

Lotul Restrâns de Informatică, Seniori, Ziua 2

Comisia științifică

May 21, 2026

Problema 1. Hello-Goodbye

Propusă de: Luca Perju Verzotti, Ecole Polytechnique

Strategia optimă se va baza pe următoarele idei

- Un prieten zice GOODBYE dacă și numai dacă este sigur că i s-a comunicat cel mai mic număr, bazat pe informația de până acum.
- Altfel, un prieten zice HELLO dacă și numai dacă nu a zis HELLO până acum și numărul comunicat lui este mai mic sau egal cu un bound calculat pe baza informației de până acum.

Urmând aceste idei, putem calcula numărul necesar de runde cu ajutorul următoarei dinamici: $DP[N][T][wait]$ = numărul necesar de runde pentru a se ridica toți prietenii de la masă în cel mai rău caz, ținând cont că mai există N prieteni la masă, având T valori posibile distincte, și că următoarele $wait$ runde nu se numără, dar nimeni dintre acești N prieteni nu are voie să zică GOODBYE în timpul lor. Pentru a înțelege mai bine cum se calculează această dinamică, vom menționa câteva tranziții, începând cu cele care au $wait > 0$. Dintr-o astfel de stare, strategia descrisă mai sus implică faptul că trebuie să alegem un bound sub care prietenii zic HELLO. Așadar, presupunem că fixăm ca toți prietenii cărora le-a fost comunicată o valoare dintre primele l cele mai mici posibile rămase să zică HELLO. Atunci, noile stări în care putem ajunge sunt:

- $DP[N][T-l][wait-1]$ (cu cost 0, dacă niciun prieten nu cunoștea un număr din cele l);
- $DP[N-1][T-l][wait-1+1]$ (cu cost 1, dacă exact un prieten cunoștea un număr din cele l , așadar el va putea zice GOODBYE după $wait-1$ runde, când îi va veni rândul);
- $DP[N-k][T-l][wait-1+l]$, $\forall k, 2 \leq k \leq N$ (cu cost l , dacă k (măcar 2) prieteni cunoșteau un număr din cele l , situație în care în cazul cel mai nefavorabil ei vor avea nevoie de l momente de timp pentru a zice toți GOODBYE, ceea ce ne induce acest cost l , dar ne și crește $wait$ cu l pentru aceste l runde în care ei vor trebui așteptați să zică GOODBYE).

Deoarece considerăm cazul cel mai nefavorabil, vom lua maximum dintre aceste 3 tranziții posibile, și vom alege bound-ul l care ne dă valoarea minimă.

Tranzițiile sunt similare și din stările cu $wait = 0$, diferența fiind că vor avea cost cu 1 mai mare, dar vor și elimina un număr în plus din cele T deoarece prietenul cu numărul minim dintre acestea ar zice direct GOODBYE.

Această dinamică, împreună cu găsirea bound-urilor optime, poate fi calculată în $O(N \cdot T^3)$, dar se pot face diferite observații precum faptul că bound-urile optime sunt monotone în dimensiunea T , care pot reduce complexitatea la $O(N \cdot T^2 \cdot \log T)$.

Problema 2. Mediana

*Propusă de: Liviu Armand Gheorghe, Universitatea București ,
Vlad-Mihai Bogdan, Universitatea București,
Tamio-Vesa Nakajima, Universitatea Marburg*

Pentru început, vom încerca să rezolvăm problema pentru o valoare x fixată - valoare cu care vom vrea să formăm o subsecvență de lungime maximă.

Vom transforma șirul a într-un nou șir, b , unde $b_i = 0$ dacă $a_i = x$, $b_i = 1$ dacă $a_i > x$ și $b_i = -1$ dacă $a_i < x$. Acum, vrem să găsim cea mai lungă subsecvență a șirului b care să conțină numai valori de zero după aplicarea operațiilor de reducere la mediană în mod optim.

O primă observație este că valorile de -1 sunt cele care ne deranjează mai mult în soluție: dacă aplicăm operația asupra valorilor $[0, 1]$, rezultatul este 0 , iar dacă aplicăm operația asupra valorilor $[-1, 0]$, rezultatul este -1 . Prin urmare, în mod intuitiv, preferăm valorile de 1 peste cele de -1 .

Așadar, dacă avem o subsecvență care conține doar valori de -1 , vom aplica operația asupra întregii subsecvențe, reducând-o la o singură valoare de -1 .

Vom nota cu d șirul obținut după compresia valorilor de -1 din șirul b .

Să numim un bloc din șirul d o subsecvență $d[i \dots j]$ cu proprietatea că $d_i = d_j = 0$ și nu există nicio valoare de 0 în interiorul blocului.

Vom clasifica blocurile în una dintre următoarele șapte categorii:

- Bloc santinelă/vid - $i + 1 = j$.
- Bloc de sumă -1 - în mod particular, acest bloc are următoarea formă: $[0, -1, 1, -1, 1, \dots, -1, 0]$.
- Bloc de sumă 1 .
- Bloc de sumă > 1 .
- Bloc de sumă zero de tipul $[-1, 1]$ - acest bloc are următoarea formă: $[0, -1, 1, -1, 1, \dots, -1, 1, 0]$.
- Bloc de sumă zero de tipul $[1, -1]$ - acest bloc are următoarea formă: $[0, 1, -1, 1, -1, \dots, 1, -1, 0]$.
- Bloc universal de sumă zero - acest bloc are suma zero, dar are atât un prefix de sumă pozitivă, cât și un prefix de sumă negativă. Prin urmare, acest bloc poate fi tratat fie ca un bloc de tipul $[-1, 1]$, fie ca un bloc de tipul $[1, -1]$, în funcție de cum este mai avantajos pentru noi.

Având aceste tipuri de blocuri, putem să mai facem observații despre structura problemei:

- Toate tipurile de blocuri, mai puțin blocul de sumă -1 , se pot reduce la o secvență de două zerouri - adică putem scăpa de toate valorile dintre cele două zerouri consecutive.

- Două blocuri vecine pot interacționa între ele cu șansa să nu pierdem nicio valoare de zero, însă două blocuri care nu sunt vecine nu pot interacționa fără să pierdem din numărul de valori de zero.
- Dacă avem un bloc de tip santinelă la poziția i în șir, blocurile din stânga acestui bloc nu pot interacționa cu blocurile din dreapta sa, fără să pierdem din numărul de valori de zero.

Prin interacțiune între două blocuri, înțelegem aplicarea unei operații care atinge elemente din ambele blocuri.

Din observațiile de mai sus, putem concluziona că un bloc de sumă mai mare decât 1 este la fel de bun ca un bloc de sumă egală cu 2, de vreme ce un bloc poate să interacționeze în mod optim cu cel mult două alte blocuri.

Prin urmare, putem să aplicăm un ultim pas de compresie, în care:

- Un bloc de sumă 1 este înlocuit cu secvența $[1]$.
- Un bloc de sumă > 1 este înlocuit cu secvența $[1, 1]$.
- Un bloc de sumă -1 este înlocuit cu secvența $[-1]$.
- Un bloc de sumă zero de tipul $[-1, 1]$ este înlocuit cu secvența $[-1, 1]$.
- Un bloc de sumă zero de tipul $[1, -1]$ este înlocuit cu secvența $[1, -1]$.
- Un bloc universal de sumă zero este înlocuit cu secvența $[0]$.

Blocurile de tip santinelă vor rămâne neschimbate, însă acestea vor delimita secvențe asupra cărora putem aplica operații. Așadar, este suficient să rezolvăm problema între două blocuri de tip santinelă.

Să analizăm cazurile în care putem să scăpăm de blocurile de sumă -1 fără să eliminăm din valorile de zero:

- $0 \ [-1] \ 0 \ [1] \ 0$ - putem aplica operația asupra $[-1, 0, 1]$.
- $0 \ [1] \ 0 \ [-1] \ 0$ - putem aplica operația asupra $[1, 0, -1]$.
- $0 \ [-1] \ 0 \ [1, 1] \ 0$ - putem aplica operația asupra $[-1, 0, 1]$.
- $0 \ [1, 1] \ 0 \ [-1] \ 0$ - putem aplica operația asupra $[1, 0, -1]$.
- $0 \ [-1] \ 0 \ [1, -1] \ 0 \ [1, -1] \ 0 \dots 0 \ [1, -1] \ 0 \ [1] \ 0$ - mereu alegem să aplicăm operația asupra primului bloc de sumă -1 și a primului 1 din blocul de sumă zero de tipul $[1, -1]$. Repetăm acest proces până la final, când putem aplica operația asupra ultimei valori de -1 și a ultimului 1 din blocul de sumă 1.
- $0 \ [1] \ 0 \ [-1, 1] \ 0 \ [-1, 1] \ 0 \dots 0 \ [-1, 1] \ 0 \ [-1] \ 0$ - aplicăm același raționament ca în cazul de mai sus.

- De vreme ce am înlocuit blocurile universale de sumă zero cu o secvență de tip $[0]$, putem întotdeauna, dacă avem o valoare de -1 , să o “mutăm” în oricare parte a unui bloc universal de sumă zero, fără să pierdem niciun zero.

Evident, aceste cazuri se pot și combina între ele, producând efecte “domino”.

Pe lângă aceste blocuri, este important să ținem cont și de ce se întâmplă înainte de primul 0, respectiv după ultimul 0 din șir (sau interval de query). Observăm că este benefic să facem operații care să atingă elemente dinaintea primei valori de 0, dacă, pornind din aceasta, putem găsi un sufix de sumă pozitivă - echivalent cu un bloc de sumă 1. Același raționament este valabil și pentru elementele de după ultimul 0, doar că în acest caz căutăm un prefix de sumă pozitivă. Prin urmare, la șirul nostru se mai pot adăuga maxim alte două blocuri de sumă 1.

Acum, putem să începem să construim o soluție: vom parcurge șirul d după ultimul pas de compresie și vom păstra două variabile: *current_sum* - suma curentă a subsecvenței pe care lucrăm la un moment dat, respectiv *underflows* - numărul de blocuri de sumă -1 pe care nu am reușit să le eliminăm fără a pierde exact un zero. După o analiză mai atentă asupra cazurilor de mai sus, un mod mai ușor de implementare se bazează pe următoarele reguli:

- Orice subsecvență de sumă zero din șirul d compresat poate să păstreze toate valorile de zero din șirul b fără să pierdem nimic.
- Dacă avem o subsecvență de sumă -1 , la care se mai adaugă un bloc de sumă -1 , este clar că nu putem “repara” subsecvența de sumă -1 fără să pierdem vreun zero. Prin urmare, valoarea *underflows* crește cu 1.
- Dacă avem o subsecvență de sumă 1, la care se adaugă un bloc de sumă 1, este clar că subsecvența de sumă 1 nu ne mai poate ajuta mai departe, deci o putem ignora și considera că am pornit o nouă subsecvență de sumă 1.

Pe baza acestor observații, ajungem la o soluție simplă: trecem prin valorile din șirul d compresat, adăugând la *current_sum* valorile curente. Dacă *current_sum* devine -2 , atunci creștem valoarea lui *underflows* cu 1. La fiecare pas, păstrăm *current_sum* în intervalul $[-1, 1]$. La final, dacă *current_sum* este -1 , atunci creștem din nou valoarea lui *underflows* cu 1.

Așadar, dacă aplicăm în mod naiv această rezolvare, obținem o soluție de complexitate $O(n^3 \cdot q)$ sau $O(n^2 \cdot q)$, în funcție de implementare.

Pentru restul de subtask-uri, mai puțin ultimul, scopul este să rezolvăm problema în complexități de genul $O(n \cdot q)$ sau $O(n \cdot q \cdot \log n)$. Pentru a optimiza soluția de mai sus, putem, de exemplu, să iterăm doar prin valorile distincte care apar în intervalul de query. Acum, vrem să rezolvăm problema pentru o valoare x fixată, în $O(cnt_x)$, unde cnt_x este numărul de apariții ale valorii x în intervalul de query.

Pentru asta, avem nevoie de o structură care poate răspunde în timp constant sau logaritmice la următoarele întrebări:

- Ce tip de bloc se formează între $occ_x[i]$ și $occ_x[i + 1]$?
- Există un prefix de sumă pozitivă începând de la $occ_x[i]$?
- Există un sufix de sumă pozitivă începând de la $occ_x[i]$?

Prin $occ_x[i]$ am notat a i -a apariție a valorii x în șir.

Putem construi acest oracol folosind o altă structură care poate suporta următoarele operații:

- Menține un șir c de valori de -1 , 0 și 1 .
- Range-Update pe intervalul $[l, r]$ - $c[l \leq i \leq r] = x$.
- Range-Sum pe intervalul $[l, r]$.
- First-Positive-Suffix pentru un l - returnează primul $i \geq l$ pentru care $\text{Range-Sum}([l, i]) > 0$.
- First-Positive-Prefix pentru un r - returnează primul $i \leq r$ pentru care $\text{Range-Sum}([i, r]) > 0$.

Un arbore de intervale poate suporta toate operațiile de mai sus în timp logaritmic (sau logaritmic la pătrat, în funcție de implementare).

Pentru a construi oracolul, este suficient să parcurgem valorile din șirul a de la cea mai mică până la cea mai mare, iar în momentul în care augmentăm structura cu valoarea v , să precalculăm și valorile de care oracolul va fi întrebat pentru această valoare v . Prin urmare, pornim cu un șir format doar din 1 , iar când trecem de la valoarea v la valoarea $v + 1$, trebuie să punem -1 pe pozițiile unde înainte aveam 0 , iar valorile de $v + 1$ devin la rândul lor 0 . Este de asemenea nevoie să ținem minte regiunile contigue de valori de -1 , lucru care se poate face în diverse moduri (unul dintre ele fiind o structură de tip union-find).

În funcție de implementare - mai exact, dacă facem această precalculare de la început, respectiv menținem o structură de tip arbore de intervale pentru fiecare query - obținem fie o soluție de complexitate $O(n \cdot q)$, fie o soluție de complexitate $O(n \cdot q \cdot \log n)$.

Ultimul subtask al problemei ne cere o soluție de complexitate $O((n+q) \cdot \sqrt{n})$. Acesta valorează doar un punct, însă implementarea este non-trivială. În cele ce urmează vom prezenta de ce avem nevoie pentru a rezolva problema în complexitatea dorită, însă detaliile de implementare sunt subtile și sunt lăsate ca exercițiu celui care rezolvă problema.

Vom folosi aceeași abordare ca mai sus, împreună cu oracolul care ne poate răspunde la interogări în timp constant. Pentru o ordine de rezolvare eficientă a query-urilor, vom folosi algoritmul lui Mo - varianta cu undo. Mai exact, vom sorta query-urile după perechea $(\lfloor \frac{l}{\sqrt{n}} \rfloor, r)$. Acum, vrem să rezolvăm toate query-urile cu capătul stâng în bucket-ul b , știind că toate acestea vor avea capătul drept în ordine crescătoare. Dacă notăm cu q' numărul de query-uri din acest bucket, vrem să rezolvăm problema în $O((n + q') \cdot \sqrt{n})$.

Abordarea este una clasică - de vreme ce capătul drept crește mereu, vom menține o structură pentru elementele care sunt în afara bucket-ului capătului stâng și care este augmentată până la capătul drept al fiecărui query. Pentru fiecare query, acum trebuie să augmentăm structura și cu elementele din bucket-ul capătului stâng, iar mai apoi să facem undo pe aceste elemente.

Prin urmare, avem nevoie de o structură care să rețină informația despre răspunsul pentru fiecare valoare x . Această structură trebuie să poată suporta operații de tipul: “adaugă un element nou la începutul șirului lui x ”, respectiv “adaugă un element nou la sfârșitul șirului lui x ”.

Prin urmare, vrem să vedem care ar fi efectul adăugării unui bloc la începutul soluției pentru valoarea x , respectiv care ar fi efectul adăugării unui bloc la sfârșitul soluției pentru valoarea x . De asemenea, un detaliu care face implementarea mai dificilă este faptul că trebuie să augmentăm soluția pentru valoarea x și cu un prefix, respectiv sufix, în unele situații.

La final, vom descrie cum putem menține o structură care să răspundă la întrebarea: “dacă aș adăuga un bloc de un anumit tip la începutul sau la finalul soluției, cum s-ar schimba suma finală, respectiv valoarea *underflows* a algoritmului?”.

Putem reține o structură care reține, pentru fiecare valoare de intrare a lui *current_sum*, care ar fi perechea (*current_sum*, *underflows*) după aplicarea operațiilor. Putem compune două astfel de structuri de stare, *a* și *b*, în felul următor:

- $(a + b).current_sum[s] = b.current_sum[a.current_sum[s]]$.
- $(a + b).underflows[s] = a.underflows[s] + b.underflows[a.current_sum[s]]$.

De menționat că este foarte importantă ordinea în care compunem aceste structuri de stare.

Notă: folosind structura de stare descrisă mai sus, putem rezolva problema în $O(n \cdot q)$, dacă rezolvăm problema pentru fiecare valoare x , iar mai apoi interogăm, pentru fiecare interval de query, care este rezultatul pe acel interval în structura valorii x . Această structură poate să fie un disjoint sparse table pentru o complexitate de forma $O(n \cdot q \cdot \log n)$ sau un disjoint sparse table combinat cu metoda celor patru ruși pentru o complexitate de forma $O(n \cdot q)$.

Problema 3. Triunghiuri

Propusă de: Andrei Chertes, Universitatea Politehnică București

Inițial vom analiza fiecare proprietate pentru fiecare punct pe care vrem să îl găsim.

Centrul de greutate al unui triunghi este media aritmetică a celor trei vârfuri. Pentru triunghiul i format din: un punct cunoscut P_i și alte două puncte necunoscute Q_{a_i} și Q_{b_i} , centru de greutate este chiar Q_i , deci

$$Q_i = \frac{P_i + Q_{a_i} + Q_{b_i}}{3}.$$

Înmulțind cu 3 obținem N ecuații liniare:

$$3Q_i = P_i + Q_{a_i} + Q_{b_i}, \quad i = 0, \dots, n-1.$$

Sistemul fiind liniar, coordonatele x și y se rezolvă independent. Vom lucra cu o coordonată, notând Q_i necunoscuta și P_i termenul liber al ecuației i . Scris matriceal:

$$MQ = P, \quad M_{i,i} = 3, \quad M_{i,a_i} = M_{i,b_i} = -1.$$

Vom analiza în continuare mai multe rezolvări ale sistemului în funcție de diferite situații. Important de menționat este peste ce corp rezolvăm sistemul. Peste $\mathbb{Q}$, numărătorii și numitorii fracțiilor depășesc cu mult orice tip de date întreg. Peste $\mathbb{R}$ (cu `double` sau `long double`), apar erori de rotunjire care, la valori până în $7 \cdot 10^{18}$, depășesc 0.5 și ne distrug precizia finală. O soluție elegantă pentru ambele probleme este să lucrăm într-un corp finit $\mathbb{Z}_P \pmod{P}$ (un număr prim). Putem să alegem un număr prim $P > 7 \cdot 10^{18}$ (de exemplu $P = 7 \cdot 10^{18} + 13$ și vom lucra pe `__int128`) sau să alegem mai multe numere prime P_i (pe `long long`) și să "combinăm" rezultatele obținute folosind *Teorema Chineză a Resturilor*.

Important este și că soluția este unică, ce am precizat și în enunț. Se poate demonstra că sistemul este unic determinat, dacă matricea M a sistemului are determinantul $\det(M) \neq 0$, care se poate demonstra folosind valori proprii și caracterizarea lor folosind *Discurile Gershgorin*.

Construim graful neorientat în care fiecare ecuație i devine o muchie între a_i și b_i . Avem N vârfuri și N muchii, iar enunțul garantează conexitatea, așadar graful este *uniciclic* - un singur ciclu, de care se agață eventuali arbori. Această structură este cheia pentru subtask-urile mari.

Subtask 1: $N = 3$

Cu trei necunoscute, sistemul se rezolvă pe foaie. Pentru $N = 3$, fiecare triunghi i are ca vârfuri necunoscute pe celelalte două, a_i și b_i . Cele trei ecuații sunt:

$$\begin{cases} 3Q_0 - Q_1 - Q_2 = P_0, \\ -Q_0 + 3Q_1 - Q_2 = P_1, \\ -Q_0 - Q_1 + 3Q_2 = P_2, \end{cases} \quad \text{adică} \quad \begin{pmatrix} 3 & -1 & -1 \\ -1 & 3 & -1 \\ -1 & -1 & 3 \end{pmatrix} \begin{pmatrix} Q_0 \\ Q_1 \\ Q_2 \end{pmatrix} = \begin{pmatrix} P_0 \\ P_1 \\ P_2 \end{pmatrix}.$$

Adunăm cele trei ecuații. Pe fiecare coloană a matricei suma coeficienților este $3 + (-1) + (-1) = 1$, așa că în partea stângă rămâne $Q_0 + Q_1 + Q_2$. Notând $S = Q_0 + Q_1 + Q_2$, obținem direct:

$$S = P_0 + P_1 + P_2.$$

Suma necunoscutelor o știm. Adunăm acum S la fiecare ecuație în parte. La ecuația i , coeficientul lui Q_i era 3, iar al fiecărei dintre celelalte două necunoscute era -1 . După adunare, coeficienții lui Q_j pentru $j \neq i$ devin 0, iar coeficientul lui Q_i devine 4:

$$4Q_i = P_i + S \implies Q_i = \frac{P_i + S}{4}.$$

Trei împărțiri și am terminat.

Subtask 2: $N \leq 300$

Aplicăm eliminare gaussiană pe matricea M , în complexitate $O(N^3)$. Folosim aritmetica modulară descrisă mai sus - fie un prim mare pe `__int128`, fie CRT cu prime mai mici pe `long long`.

Subtask 3: $N \leq 3000$

Observăm că matricea M este foarte rară: doar trei elemente nenule pe linie (un 3 pe diagonală și doi -1). Pentru a profita de această structură, modificăm metoda de pivotare.

La fiecare pas, alegem ca pivot *linia cu cele mai puține elemente nenule* din coloana curentă (dintre liniile încă neprocesate). Când eliminăm o coloană, scădem un multiplu al liniei pivot din toate celelalte linii care au element nenul în acea coloană. Această operație poate transforma poziții nule în poziții nenule. Cu cât linia pivot are mai puține elemente nenule, cu atât numărul de poziții nou create este mai mic, iar matricea rămâne rară pe parcursul eliminării. Cu pivotarea aceasta, complexitatea efectivă scade la $O(N^2)$.

Subtask 4: $0 \leq \text{coordonatele } P_i, Q_i < 10^9$, pentru $0 \leq i < N$

Rescriem ecuația izolând necunoscuta din stânga:

$$Q_i = \frac{P_i + Q_{a_i} + Q_{b_i}}{3}.$$

Dacă am ști Q_{a_i} și Q_{b_i} , am putea calcula ușor Q_i .

În plus, spunem despre Q_i că este "echilibrat", dacă pentru Q_i , Q_{a_i} și Q_{b_i} fixate, egalitatea de mai sus este adevărată. Sistemul este rezolvat când toate Q_i sunt echilibrate.

Pornim de la o configurație care *nu* este în echilibru - de exemplu $Q = 0$ peste tot. Pentru fiecare componentă i , Q_i nu coincide cu media vecinilor săi. O putem "echilibra": înlocuim Q_i cu media corectă,

$$Q_i \leftarrow \frac{P_i + Q_{a_i} + Q_{b_i}}{3}.$$

Aplicăm aceeași operație simultan la toate componentele. Asta definește o *transformare de echilibrare* pe vectorul Q :

$$T(Q_i) = \frac{P_i + Q_{a_i} + Q_{b_i}}{3}.$$

Algoritmul: aplicăm T repetat. Pornim cu $Q^{(0)} = 0$ și calculăm $Q^{(k+1)} = T(Q^{(k)})$. Vrem să ajungem ca $Q = T(Q)$.

Vom demonstra mai târziu că aplicând T de un număr de ori, ajungem foarte aproape de soluția sistemului.

Condiția de oprire a aplicărilor poate fi: $|Q^{(k)} - Q^{(k+1)}| < \epsilon$, sau depășirea unui număr fixat de aplicări.

Se observă că în acest caz nu mai putem lucra peste $\mathbb{Z}_P$, ci va trebui să lucrăm peste $\mathbb{R}$ (cu `double` sau `long double`) pentru că dorim să știm cât de aproape suntem de soluție. Din această cauză apar erori de precizie care fac ca această soluție să funcționeze doar pe acest subtask.

Subtask 5: N - impar, laturile $Q_{a_i}Q_{b_i}$ formează un poligon

La eliminarea gaussiană am lucrat modulo un prim mare pentru a evita fracțiile. Nu toate primele sunt la fel de utile - alegerea depinde de ce vrem să facem cu rezultatul. Aici alegem aproape cel mai mic prim posibil: $P = 3$. Atunci toate operațiile se reduc la calcule pe cifre $\{0, 1, 2\}$, iar coeficientul 3 din ecuația noastră are un rol important.

Din enunț avem N ecuații, fiecare legând trei necunoscute Q_i, Q_{a_i}, Q_{b_i} . Asta se traduce natural într-un graf, dar cum îl construim? Cea mai utilă alegere e să atribuim ecuația i unei muchii între a_i și b_i - anume cele două necunoscute care apar în partea dreaptă a ecuației.

Privim ecuația

$$3Q_i = P_i + Q_{a_i} + Q_{b_i}$$

modulo 3. Termenul $3Q_i$ se anulează, iar relația devine

$$Q_{a_i} + Q_{b_i} \equiv -P_i \pmod{3}.$$

Observăm contrastul cu soluția prin echilibrare: acolo, ecuația ne dădea o formulă pentru a calcula Q_i din vecinii săi; aici, modulo 3, ecuația *elimină* Q_i și ne lasă cu o relație doar între cei doi vecini ai săi. Asta se potrivește exact cu felul în care am atribuit ecuațiile la muchii: pe muchia i , suma cifrelor capetelor este $-P_i \pmod{3}$, fără referire la Q_i . Necunoscutele rămase sunt cifrele $r_j = Q_j \pmod{3} \in \{0, 1, 2\}$.

Cu ecuațiile așezate pe muchii și cifrele așezate pe noduri, structura grafului este utilă: cunoscând cifra unui vârf, ecuația oricărei muchii incidente îi determină pe cealaltă.

Notăm nodurile ciclului $v_0, v_1, \dots, v_{L-1}, v_0$ și muchiile $e_0, \dots, e_{L-1}$. Pornind cu r_{v_0} liber:

$$r_{v_1} = -P_{e_0} - r_{v_0}, \quad r_{v_2} = -P_{e_1} + P_{e_0} + r_{v_0}, \dots$$

La revenirea în v_0 , semnul lui r_{v_0} s-a schimbat de L ori, deci

$$r_{v_0} \equiv (-1)^L r_{v_0} + \sum_{i=0}^{L-1} (-1)^{L-1-i} (-P_{e_i}) \pmod{3}.$$

Pentru L impar, $(-1)^L = -1$, iar relația devine $2r_{v_0} \equiv \text{constantă} \pmod{3}$. Cum $2^{-1} \equiv 2 \pmod{3}$, r_{v_0} se determină unic. Restul cifrelor se obțin prin propagare.

Pentru a afla Q complet, scriem $Q_i = r_i + 3\tilde{Q}_i$ și înlocuim în ecuația originală și împărțim la 3:

$$3\tilde{Q}_i = \tilde{P}_i + \tilde{Q}_{a_i} + \tilde{Q}_{b_i}, \quad \tilde{P}_i = \frac{P_i - 3r_i + r_{a_i} + r_{b_i}}{3}.$$

Se observă că, împărțirea la 3 se face exact deoarece noi am calculat r astfel încât $P_i - 3r_i + r_{a_i} + r_{b_i} \equiv 0 \pmod{3}$.

Noul sistem are aceeași formă, doar pe $\tilde{Q}$. Repetăm extragerea cifrei mod 3. Practic vom afla fiecare cifra în baza 3 a soluției. Cum $Q_i < 7 \cdot 10^{18} < 3^{40}$, 40 pași sunt suficienți pentru a extrage toate cifrele. Complexitate $O(N)$, cu constanta 40.

Subtask 6: Date de intrare generate aleator

Pe teste generate aleator, lungimea ciclului L poate avea orice paritate. Dacă L este par, termenul $(-1)^L$ devine $+1$. Astfel, rămânem cu o condiție de “verificare”:

$$\sum_{i=0}^{L-1} (-1)^i P_{e_i} \equiv 0 \pmod{3} \quad (*)$$

Încercăm pe rând $r_{v_0} \in \{0, 1, 2\}$. Pentru fiecare r_{v_0} fixat, propagăm, trecem la următorul nivel, și verificăm (*) la nivelul nou. Dacă (*) nu se verifica, trecem înapoi. Pentru date de intrare generate aleator, dacă r_{v_0} este ales greșit la nivelul k , atunci (*) nu se va verifica la un nivel apropiat de k (eventual nivelul $k + 1$). Pe testele adversariale însă, backtracking-ul se poate comporta exponențial.

Soluția de la subtask-ul 4 poate fi îmbunătățită pentru a rezolva și acest subtask.

Subtask 7: Fără restricții suplimentare

Pentru cazul general avem mai multe soluții care se bazează pe soluțiile de la subtask-urile precedente.

Continuând ideea de la ciclu impar, vrem să extragem cifrele lui Q una câte una. Pe ciclu par, problema este că la fiecare nivel avem trei candidate pentru cifre. Avem nevoie de o relație suplimentară care să decidă cifra corectă.

Definim:

- Colorarea nodurilor: $\text{col}_{a_e} \cdot \text{col}_{b_e} = -1$ pentru orice muchie e - adică col este colorarea bipartită a grafului.
- Colorarea muchiilor: $\mu_{e_1} \cdot \mu_{e_2} = -1$ pentru oricare două muchii e_1, e_2 incidente la un nod de pe ciclu. Pe arborii agățați $\mu = 0$.

Această alegere pe ciclu este unică (până la semn): μ alternează $+1, -1, +1, -1, \dots$ pe muchiile ciclului în ordinea parcurgerii. La fiecare nod de pe ciclu, cele două muchii incidente au μ opuse, deci $\mu_{e_1} + \mu_{e_2} = 0$.

Înmulțind ecuația $r_{a_i} + r_{b_i} \equiv -P_i \pmod{3}$ cu μ_i și sumând pe ciclu:

$$\sum_{e \in \text{cyc}} \mu_e (r_{a_e} + r_{b_e}) \equiv - \sum_{e \in \text{cyc}} \mu_e P_e \pmod{3}.$$

Membrul stâng, rescris pe noduri, devine $\sum r_v (\mu_{e_1(v)} + \mu_{e_2(v)}) = 0$ - termenii cu r se anulează. Rămâne

$$\sum_{e \in \text{cyc}} \mu_e P_e \equiv 0 \pmod{3},$$

care este chiar $(*)$. Notăm $A = (\sum \mu_e P_e)/3$, un întreg cunoscut. Vrem ca după aflarea cifrei r și trecerea la $\tilde{P}$, condiția să se păstreze:

$$\sum_e \mu_e \tilde{c}_e \equiv 0 \pmod{3}.$$

Înmulțind cu 3 și folosind $\tilde{P}_e = (P_e - 3r_e + r_{a_e} + r_{b_e})/3$:

$$3 \sum_e \mu_e \tilde{P}_e = \sum_e \mu_e P_e - 3 \sum_e \mu_e r_e + \sum_e \mu_e (r_{a_e} + r_{b_e}).$$

Ultima sumă o rescriem pe noduri:

$$\sum_e \mu_e (r_{a_e} + r_{b_e}) = \sum_v r_v (\mu_{e_1(v)} + \mu_{e_2(v)}) = 0,$$

prin alegerea lui μ . Deci

$$3 \sum_e \mu_e \tilde{P}_e = \sum_e \mu_e P_e - 3 \sum_e \mu_e r_e,$$

iar după împărțirea cu 3:

$$\sum_e \mu_e \tilde{P}_e = A - \sum_e \mu_e r_e.$$

Condiția $\sum_e \mu_e \tilde{P}_e \equiv 0 \pmod{3}$ devine:

$$A - \sum_e \mu_e r_e \equiv 0 \pmod{3}. \quad (\diamond)$$

Aceasta este relația pe cifre pe care o căutam.

Calculăm întâi o soluție particulară a sistemului mod 3, fixând $r_{v_0} = 0$ și propagând prin graf. Rezultatul este un vector B cu $B_{v_0} = 0$. Cele trei soluții la nivelul curent sunt:

$$r = B + t \cdot \text{col}, \quad t \in \{0, 1, 2\}.$$

Justificarea: pentru fiecare muchie e ,

$$r_{a_e} + r_{b_e} = (B_{a_e} + t \text{col}_{a_e}) + (B_{b_e} + t \text{col}_{b_e}) = B_{a_e} + B_{b_e} + t(\text{col}_{a_e} + \text{col}_{b_e}) = B_{a_e} + B_{b_e},$$

pentru că $\text{col}_{a_e} + \text{col}_{b_e} = 0$ pe orice muchie (bipartit). Deci $B + t \cdot \text{col}$ satisface ecuațiile la fel ca B pentru orice t .

Substituind în ($\diamond$):

$$A - \sum_e \mu_e (B_e + t \text{col}_e) \equiv 0 \pmod{3},$$

sau, notând $C = \sum_e \mu_e B_e$ și $D = \sum_e \mu_e \text{col}_e$:

$$A - C - tD \equiv 0 \pmod{3} \implies t \equiv (A - C) \cdot D^{-1} \pmod{3}.$$

Aceasta determină t unic, cu condiția ca $D \not\equiv 0 \pmod{3}$.

Pentru un ciclu de lungime L multiplu de 3, vectorul μ alternant produce $D = \pm L \equiv 0 \pmod{3}$, iar inversul D^{-1} nu există. Trebuie să înlocuim μ cu un vector μ' care: **(a)** păstrează proprietatea la noduri (suma valorilor μ' pe muchiile incidente la orice nod este divizibilă cu 3); **(b)** are $D' \not\equiv 0 \pmod{3}$.

Naiv, $\mu' = \mu + 3v$ pentru un vector v pe muchii ne dă același rest mod 3. Pentru a schimba comportamentul mod 3, împărțim cu 3 după adăugare:

$$\mu'_e = \frac{\mu_e + 3v_e - b_e}{3},$$

unde b_e este un termen ales astfel încât împărțirea să fie exactă și μ' să rămână în spațiul valid. Algebra impune alegerea $b_e =$ suma valorilor v pe muchiile incidente la vârful asociat lui e . Cu această alegere, μ' păstrează proprietatea pe nod, iar

$$D' = D/3 + \sum_e v_e \text{col}_e.$$

Pentru ca $(\mu_e + 3v_e - b_e)/3$ să fie întreg la fiecare e , v trebuie să satisfacă $b_e \equiv \mu_e \pmod{3}$ peste tot - o ecuație pe v pe muchiile grafului.

Această ecuație se rezolvă în $O(N)$ pe graf uniciclic, prin eliminarea treptată a frunzelor. La fiecare frunză, muchia incidentă are valoarea v fixată unic; o eliminăm și mergem mai departe. Rămânem cu ciclul pe care îl parcurgem cu recurența $v_{\text{următor}} = \mu_v - v_{\text{anterior}} \pmod{3}$.

Repetăm transformarea $\mu \rightarrow \mu'$ până când $D \not\equiv 0 \pmod{3}$. La fiecare aplicare D se împarte cu 3 (din formula de mai sus), deci numărul de iterații este valoarea exponentului lui 3 din descompunerea a lui D inițial.

Complexitate timp: $O(N)$ cu constanta 40 după pre-calculare.

În plus, observăm ca dacă am fi știut valoarea unei singure necunoscute am putea folosi rezolvarea de la Subtask 5 pentru a rezolva acest subtask. Problema este cum aflăm această valoare? Lăsăm cititorul să găsească o soluție.

A doua soluție evită complet aritmetica modulară (și multă matematică din spatele ei). Pornește de la observația că, deși nu cunoaștem Q_i , îl putem încadra între două margini:

$$\ell_i \leq Q_i \leq h_i.$$

Inițializăm $\ell_i = 0$ și $h_i = 7 \cdot 10^{18}$. Vom restrânge ambele margini pas cu pas până devin egale.

Din $3Q_i = P_i + Q_{a_i} + Q_{b_i}$ și $Q_{a_i} \geq \ell_{a_i}$, $Q_{b_i} \geq \ell_{b_i}$, obținem

$$Q_i \geq \frac{P_i + \ell_{a_i} + \ell_{b_i}}{3}.$$

Cum Q_i este întreg, putem rotunji în sus împărțirea cu 3 fără a strica marginea inferioară:

$$\ell_i \leftarrow \max\left(\ell_i, \left\lceil \frac{P_i + \ell_{a_i} + \ell_{b_i}}{3} \right\rceil\right).$$

Simetric, $Q_{a_i} \leq h_{a_i}$ și $Q_{b_i} \leq h_{b_i}$ produc o margine superioară, rotunjită în jos:

$$h_i \leftarrow \min\left(h_i, \left\lfloor \frac{P_i + h_{a_i} + h_{b_i}}{3} \right\rfloor\right).$$

Atenție la utilizarea (necesară) min / max!

Dacă notăm cu L lungimea celui mai mare interval de la pasul precedent $[\ell'_i, h'_i]$ observăm următoarea inegalitate pentru intervalele $[\ell_i, h_i]$ de la pasul curent:

$$h_i - \ell_i \leq \left\lfloor \frac{h'_{a_i} + h'_{b_i} + P_i}{3} \right\rfloor - \left\lceil \frac{\ell'_{a_i} + \ell'_{b_i} + P_i}{3} \right\rceil \leq \frac{h'_{a_i} + h'_{b_i} + P_i}{3} - \frac{\ell'_{a_i} + \ell'_{b_i} + P_i}{3}$$

Dacă grupăm:

$$h_i - \ell_i \leq \frac{(h'_{a_i} - \ell'_{a_i}) + (h'_{b_i} - \ell'_{b_i})}{3}$$

Deci:

$$h_i - \ell_i \leq \frac{2L}{3}$$

Asta înseamnă că lungimea celui mai mare interval de la pasul curent este cel mult două treimi din lungimea celui mai mare interval de la pasul precedent.

Deci putem face aproximativ 110 de pași pentru a ajunge de la un interval de lungime $7 \cdot 10^{18}$ la un interval de lungime 1.

O singură margine. Update-urile pentru ℓ și h sunt independente: ℓ depinde doar de ℓ -uri, h doar de h -uri. Cele două margini sunt două probleme paralele care converg ambele la Q . Este suficient să ținem doar una. Doar cu ℓ , ne oprim când verificarea $3\ell_i = P_i + \ell_{a_i} + \ell_{b_i}$ se satisface peste tot.

Complexitate timp: $O(N)$ cu constanta 110.

Echipa

Problemele pentru acest baraj au fost pregătite de:

- Adrian Panaete, Colegiul Național "A.T. Laurian", Botoșani
- Nicoli Marius, Colegiul Național "Frații Buzești" Craiova
- Frâncu Cătălin, Nerdvana
- Tamio-Vesa Nakajima, Universitatea Marburg, Hesse
- Andrei-Costin Constantinescu, ETH, Zürich, Elveția
- Alexandru Lorintz, Universitatea "Babeș-Bolyai", Cluj-Napoca
- Toma Ariciu, Universitatea Politehnică București
- Andrei-Cristian Ivan, Universitatea Politehnică București
- Dan-Constantin Spătărel, București
- Vlad-Mihai Bogdan, Universitatea București
- Eugenie-Daniel Posdărăscu, Youni, București
- Radu-Teodor Prosie, Universitatea București
- Liviu Armand Gheorghe, Universitatea București
- Tiberiu Cozma, Universitatea Alexandru Ioan Cuza, Iași
- Ion Andrei Robert, Ecole Polytechnique, Paris
- Cismaru Mihaela, Google Paris
- Perju Verzotti Luca, Ecole Polytechnique, Paris
- Gheorghies Alexandru, Universitatea "Alexandru Ioan Cuza" Iași
- Boacă Andrei, Universitatea "Alexandru Ioan Cuza" Iași
- Daniel Gheorghe, Universitatea din București
- Arsenoiu Iulian, Columbia University
- Feodorov Andrei, ETH, Zürich, Elveția
- Botnaru Victor, Universitatea Politehnica Bucuresti, ACS
- Popescu Adrian, Universitatea Politehnica Bucuresti, ACS
- Măgureanu Livia, JetBrains
- Popa Bogdan-Ioan, Just 4 Kids
- Andrei Chertes, Universitatea Politehnică București